

Guide de lecture du code AI Riviera 2

Document compagnon pour comprendre l'architecture, les fichiers et les fonctions principales.

Ce document est fait pour te remettre de l'ordre dans la tête quand le projet semble trop gros.

L'idée principale: **AI Riviera 2 est un chatbot documentaire**. Il prend des documents publics de La Tour-de-Peilz, les transforme en textes et données, les indexe dans une base, puis permet de poser des questions dessus avec des sources visibles.

1. La carte mentale du projet

```
scrape-la-tour-de-peilz/  
  Recupere les documents publics depuis le site officiel.  
  Extrait les textes des PDF ou pages HTML.  
  Cree des fichiers .txt et .json dans documents/.
```

```
documents/  
  Contient les documents deja extraits:  
  - .txt = texte lisible  
  - .json = metadonnees du document
```

```
data/  
  Contient des exports ou donnees structurees:  
  seances, documents, objets politiques, statistiques.
```

```
app/  
  Contient l'application:  
  - interface Streamlit  
  - ingestion  
  - recherche  
  - reponse LLM  
  - stockage Postgres / OpenSearch / SQLite
```

```
db/  
  Contient la structure SQL de la base.
```

Le coeur du projet est dans app/.

Les scripts dans scrape-la-tour-de-peilz/ servent surtout a fabriquer les donnees de depart.

2. Le parcours complet d'une question

Quand un utilisateur pose une question dans l'interface:

```
app/ui.py  
  recoit la question
```

```
app/structured.py  
  essaie de repondre directement avec les donnees structurees  
  exemple: "combien de motions en 2024 ?"
```

```
app/answer.py  
  peut reformuler la question pour la recherche
```

```
app/retrieval.py  
  cherche les passages les plus pertinents
```

```
app/answer.py  
  envoie les passages a Mistral ou OpenAI si une cle existe  
  sinon affiche les meilleurs extraits
```

```
app/ui.py
    affiche la reponse et les sources
```

Version ultra simple:

Question -> Recherche -> Sources -> Reponse -> Affichage

3. Les fichiers les plus importants

app/ui.py

C'est l'interface visible. Elle utilise Streamlit.

Elle fait:

- afficher le titre, les onglets et la zone de chat;
- stocker l'historique de conversation;
- recevoir les questions;
- appeler la recherche et la generation de reponse;
- afficher les sources.

Fonctions importantes:

Fonction	Role simple
<code>admin_tabs_enabled()</code>	Regarde si les onglets d'administration doivent etre affiches.
<code>current_filters()</code>	Retourne les filtres appliques a la recherche. Ici: ville = La Tour-de-Peilz.
<code>cacheable_filters()</code>	Transforme les filtres en format utilisable par le cache Streamlit.
<code>normalize_follow_up_text()</code>	Nettoie une question pour detecter si c'est une question de suivi.
<code>looks_like_follow_up()</code>	Devine si la question depend de la conversation precedente. Exemple: "et en 2025 ?"
<code>compact_history_for_question()</code>	Resume les derniers messages pour donner du contexte a une question de suivi.
<code>contextualize_question()</code>	Transforme une question courte en question complete avec l'historique.
<code>ensure_index_ready()</code>	Verifie si Postgres ou OpenSearch est pret.
<code>group_results_by_document()</code>	Regroupe plusieurs passages venant du meme document.
<code>source_link()</code>	Cree un lien Markdown vers la source PDF.
<code>link_source_mentions()</code>	Transforme "Source 1" en lien cliquable dans l'interface.
<code>render_sources()</code>	Affiche la liste des documents utilises et les passages retrouves.
<code>cached_answer_question()</code>	Fonction centrale cachee: structure, recherche, reranking, reponse.
<code>answer_question()</code>	Fonction principale appelee quand l'utilisateur pose une question.
<code>queue_question()</code>	Met une question en attente dans l'etat Streamlit.

La fonction la plus importante ici est:

```
answer_question(question, messages)
```

Elle:

1. verifie que l'index est pret;
2. ajoute le contexte si la question est une suite;
3. appelle `cached_answer_question`;
4. enregistre un diagnostic;
5. retourne la reponse et les sources.

app/answer.py

C'est le fichier qui parle aux LLM: Mistral ou OpenAI.

Il ne cherche pas les documents lui-meme. Il prend des resultats deja trouves et construit une reponse.

Fonctions importantes:

Fonction	Role simple
<code>get_secret()</code>	Lit une cle API depuis les variables d'environnement ou les secrets Streamlit.
<code>document_key()</code>	Trouve une cle stable pour identifier un document.
<code>group_results_by_document()</code>	Regroupe les passages par document.
<code>build_context()</code>	Fabrique le bloc de texte envoye au LLM avec les sources et passages.
<code>answer_with_openai()</code>	Demande a OpenAI de repondre a partir des extraits.
<code>answer_with_mistral()</code>	Demande a Mistral de repondre a partir des extraits.
<code>short_openai_completion()</code>	Petit appel OpenAI pour une tache courte: reformuler, router, reranker.
<code>short_mistral_completion()</code>	Meme idee pour Mistral.
<code>rewrite_query_with_openai()</code>	Reformule une question pour la recherche avec OpenAI.
<code>rewrite_query_with_mistral()</code>	Reformule une question pour la recherche avec Mistral.
<code>rewrite_query_with_llm()</code>	Choisit Mistral/OpenAI/auto pour reformuler.
<code>route_intent_with_openai()</code>	Demande a OpenAI si la question est structured ou rag.
<code>route_intent_with_mistral()</code>	Meme idee avec Mistral.
<code>route_intent_with_llm()</code>	Choisit le routeur actif.
<code>result_rerank_candidate()</code>	Prepare un extrait sous forme courte pour le reranking.
<code>parse_rerank_ids()</code>	Lit la liste JSON d'identifiants choisie par le LLM.
<code>rerank_results_with_llm()</code>	Demande au LLM de trier les meilleurs extraits.
<code>test_mistral_connection()</code>	Teste si Mistral repond correctement.
<code>answer_with_llm()</code>	Choisit OpenAI ou Mistral selon la configuration.
<code>llm_status()</code>	Donne l'etat du fournisseur LLM actif.
<code>answer_from_sources()</code>	Produit la reponse finale a partir des sources.
<code>_sources_section()</code>	Ajoute la liste des sources en bas de reponse.

La logique importante:

Si une cle Mistral/OpenAI existe:
le LLM fait une synthese avec les sources.

Sinon:
l'application affiche directement les meilleurs passages.

Le SYSTEM_PROMPT est tres important. Il dit au modele:

- repondre uniquement avec les extraits fournis;
- dire clairement quand les sources ne suffisent pas;
- citer les sources;
- ne pas confondre vote communal et votation populaire.

app/retrieval.py

C'est le fichier de recherche documentaire.

Son role: prendre une question et retourner une liste de passages pertinents.

Fonctions:

Fonction	Role simple
<code>tokenize()</code>	Decoupe une question en mots utiles et retire les mots vides.
<code>is_broad_legislature_query()</code>	Detecte les questions larges sur la legislature.
<code>is_council_vote_query()</code>	Detecte les questions sur les votes du Conseil communal.
<code>is_council_regulation_query()</code>	Detecte les questions sur le reglement ou les articles.
<code>load_chunks()</code>	Charge les anciens chunks JSON si on utilise le mode legacy.
<code>search()</code>	Fonction principale de recherche.
<code>merge_postgres_results()</code>	Fusionne resultats par mots-cles et resultats vectoriels.

La fonction principale:

```
search(query, limit=6, filters=None)
```

Elle fait plusieurs choses:

1. elle tokenise la question;
2. elle ajoute des filtres automatiques;
 - "motion" -> filtre motions;
 - "postulat" -> filtre postulats;
 - "article 96" -> filtre reglement;
 - "2025" -> filtre annee;
3. elle essaie la recherche hybride;
4. elle essaie Postgres mots-cles + vecteurs;
5. elle peut retomber sur SQLite ou JSON ancien mode.

Important: ce fichier est le carrefour entre la question humaine et les bases de recherche.

app/ingestion_pipeline.py

C'est le pipeline moderne d'indexation.

Son role: lire les fichiers .txt et .json dans documents/, les decouper en morceaux, creer les embeddings, puis stocker le tout.

Fonctions:

Fonction	Role simple
<code>chunk_text()</code>	Decoupe un long texte en morceaux de 1200 caracteres environ.
<code>chunks_for_document()</code>	Choisit comment decouper un document. Les articles de reglement restent entiers.
<code>load_metadata()</code>	Charge le <code>.json</code> a cote du <code>.txt</code> et enrichit les metadonnees.
<code>iter_text_files()</code>	Parcourt tous les fichiers <code>.txt</code> utiles.
<code>_document_payload()</code>	Prepare les donnees d'un document avant insertion en base.
<code>_searchable_text()</code>	Ajoute titre, auteurs, annee, statut, etc. devant chaque chunk pour ameliorer la recherche.
<code>ingest_documents()</code>	Fonction centrale qui indexe tous les documents.
<code>main()</code>	Permet de lancer le fichier avec <code>python -m app.ingestion_pipeline</code> .

La fonction centrale:

`ingest_documents()`

Elle:

1. verifie que le schema Postgres existe;
2. ouvre une connexion Postgres;
3. parcourt tous les `.txt`;
4. charge les metadonnees;
5. calcule un hash pour savoir si le document a change;
6. saute les documents inchanges;
7. decoupe le texte;
8. cree les embeddings;
9. insere les chunks dans Postgres;
10. indexe aussi dans OpenSearch si disponible;
11. journalise les statistiques.

app/postgres_store.py

C'est la couche d'accès Postgres.

Elle cache les details SQL du reste de l'application.

Fonctions importantes:

Fonction	Role simple
<code>DocumentRecord</code>	Structure Python qui represente un document avant insertion.
<code>_connect()</code>	Ouvre une connexion Postgres.
<code>ensure_schema()</code>	Cree/verifie les tables SQL.
<code>sha256_text()</code>	Calcule un hash stable d'un texte.
<code>canonical_source_url()</code>	Choisit l'URL source officielle d'un document.
<code>canonical_document_date()</code>	Trouve la meilleure date du document.
<code>canonical_fetch_date()</code>	Trouve la date de collecte.

Fonction	Role simple
<code>build_document_hash()</code>	Cree les empreintes du document et du contenu.
<code>upsert_document()</code>	Insere ou met a jour un document.
<code>get_document_by_source_url()</code>	Retrouve un document par URL.
<code>ready()</code>	Verifie si Postgres est utilisable.
<code>vector_search_ready()</code>	Verifie si la recherche vectorielle est disponible.
<code>_vector_literal()</code>	Convertit un vecteur Python en texte SQL.
<code>_normalize_search_rows()</code>	Transforme les lignes SQL en resultats utilisables par l'app.
<code>_filter_sql()</code>	Transforme les filtres Python en conditions SQL.
<code>search_chunks()</code>	Recherche par mots-cles dans Postgres.
<code>search_vector_chunks()</code>	Recherche vectorielle dans Postgres.
<code>insert_chunks()</code>	Insere les morceaux de texte indexes.
<code>delete_chunks_for_document()</code>	Supprime les anciens chunks d'un document.
<code>start_ingestion_run()</code>	Cree une ligne de debut d'ingestion.
<code>finish_ingestion_run()</code>	Marque la fin d'une ingestion.
<code>log_ingestion_event()</code>	Ajoute un evenement de log d'ingestion.

app/structured.py

C'est le gros fichier qui repond sans LLM quand la question est calculable.

Exemples:

- combien de postulats en 2024 ?
- qui a depose telle interpellation ?
- quels objets parlent de mobilite ?
- que dit l'article 96 ?
- quels montants apparaissent dans le budget ?

Il contient beaucoup de petites fonctions de detection:

`wants_...`

Ces fonctions veulent dire:

"Est-ce que la question demande ce type de reponse ?"

Exemples:

Fonction	Role simple
<code>wants_financial_question()</code>	Detecte une question financiere.
<code>wants_regulation_question()</code>	Detecte une question sur le reglement.
<code>wants_person_deposits()</code>	Detecte une question sur les depots d'une personne.
<code>wants_count_by_party()</code>	Detecte une question de comptage par parti.
<code>wants_objects_by_status()</code>	Detecte une question sur les objets par statut.
<code>wants_themed_political_objects()</code>	Detecte une question sur un theme: mobilite, bus, ecole, etc.

Puis il contient des fonctions:

`answer_...`

Ces fonctions fabriquent vraiment la reponse.

Exemples:

Fonction	Role simple
<code>answer_financial_db()</code>	Repond avec les donnees financieres en base.
<code>answer_regulation_db()</code>	Repond avec les articles du reglement.
<code>answer_object_author_db()</code>	Trouve l'auteur d'un objet politique.
<code>answer_person_deposits_db()</code>	Liste les objets deposes par une personne.
<code>answer_coauthors_db()</code>	Trouve les coauteurs.
<code>answer_count_by_party_db()</code>	Compte les objets par parti.
<code>answer_objects_by_status_db()</code>	Liste les objets selon leur statut.
<code>answer_themed_objects_db()</code>	Liste les objets lies a un theme.
<code>answer_objects_by_year_db()</code>	Liste les objets d'une annee.
<code>answer_most_deposits()</code>	Trouve qui a le plus depose.
<code>answer_deposits_by_year()</code>	Donne les depots par annee.
<code>answer_latest_deposits()</code>	Donne les derniers depots.
<code>answer_structured_question()</code>	Point d'entree final de ce fichier.

La fonction la plus importante:

`answer_structured_question(question)`

Elle essaie plusieurs strategies de reponse directe avant de passer au RAG.

`app/metadata_enrichment.py`

Ce fichier enrichit les metadonnees des documents.

Il sert a transformer des JSON parfois incomplets en metadonnees plus utiles pour la recherche.

Exemples de choses ajoutees:

- categorie;
- type de contenu;
- legislature;
- auteurs;
- parti politique;
- dates;
- references legales;
- facettes de recherche;
- statut d'un objet politique.

Fonction centrale:

`enrich_metadata(metadata, text_path=None, content=None)`

Elle prend des metadonnees brutes et renvoie des metadonnees plus propres.

`app/embeddings.py`

Ce fichier cree les vecteurs de recherche.

Fonctions:

Fonction	Role simple
<code>_normalize_text()</code>	Nettoie le texte avant embedding.
<code>_hash_embedding()</code>	Cree un faux embedding stable si OpenAI n'est pas disponible.
<code>_openai_client()</code>	Cree le client OpenAI.
<code>embed_texts()</code>	Cree les embeddings pour plusieurs textes.
<code>embed_text()</code>	Cree un embedding pour un seul texte.
<code>cosine_similarity()</code>	Mesure la proximite entre deux vecteurs.

Important: si aucune cle OpenAI n'est disponible, le code peut quand meme creer des embeddings de fallback avec hash. Ce n'est pas aussi bon, mais ca permet de fonctionner.

app/opensearch_store.py

C'est la couche OpenSearch.

Fonctions:

Fonction	Role simple
<code>get_client()</code>	Cree le client OpenSearch.
<code>load_index_body()</code>	Lit la configuration de l'index.
<code>ensure_index()</code>	Cree l'index si besoin.
<code>ensure_runtime_mapping()</code>	Verifie certains champs calcules.
<code>ready()</code>	Dit si OpenSearch est disponible.
<code>delete_document()</code>	Supprime un document de l'index.
<code>index_chunks()</code>	Envoie les chunks dans OpenSearch.
<code>_filters_to_query()</code>	Convertit les filtres en requete OpenSearch.
<code>keyword_search()</code>	Recherche par mots-cles.
<code>knn_search()</code>	Recherche vectorielle.
<code>_normalize_hit()</code>	Convertit un resultat OpenSearch en format commun.

app/hybrid_search.py

Ce fichier fusionne plusieurs types de recherche.

Fonctions:

Fonction	Role simple
<code>_merge_candidates()</code>	Fusionne resultats mots-cles et vectoriels.
<code>search_hybrid()</code>	Lance recherche hybride dans OpenSearch.
<code>grouped_sources()</code>	Regroupe les resultats par source.

app/reranker.py

Ce fichier retreie les chunks apres recherche.

Fonctions:

Fonction	Role simple
<code>canonical_priority()</code>	Donne plus de poids aux sources canoniques.
<code>regulation_priority()</code>	Donne plus de poids aux articles pertinents du reglement.
<code>rerank_chunks()</code>	Retrie les passages trouves.

app/year_metadata.py

Ce fichier normalise les annees et dates.

Il evite une confusion importante:

```

annee du fichier PDF
annee de publication
annee de l'objet politique
annee de la seance

```

Fonctions importantes:

Fonction	Role simple
<code>infer_object_year()</code>	Trouve l'annee de l'objet politique.
<code>infer_document_year()</code>	Trouve l'annee du document.
<code>infer_publication_date()</code>	Trouve la date de publication.
<code>normalize_year_metadata()</code>	Met tout ca au propre dans les metadonnees.

app/people_index.py

Ce fichier construit un index des personnes.

Il sert a relier:

- auteurs;
- motionnaires;
- postulants;
- interpellateurs;
- partis;
- objets politiques.

Classe et fonctions importantes:

Element	Role simple
<code>PersonAccumulator</code>	Accumule les infos trouvees sur une personne.
<code>normalize_name()</code>	Nettoie un nom.
<code>person_key()</code>	Cree une cle stable pour identifier une personne.
<code>alias_keys()</code>	Cree des variantes du nom.
<code>iter_author_entries()</code>	Lit les auteurs dans les metadonnees.
<code>build_people()</code>	Construit l'index en memoire.
<code>people_rows()</code>	Prepares les lignes SQL.
<code>upsert_people()</code>	Insere/met a jour les personnes en base.
<code>rebuild_people_index()</code>	Reconstruit tout l'index personnes.

app/political_objects_index.py

Ce fichier construit l'index des objets politiques.

Un objet politique peut être:

- motion;
- postulat;
- interpellation;
- preavis;
- communication;
- rapport;
- réponse.

Classe et fonctions importantes:

Element	Role simple
PoliticalObjectAccumulator	Accumule les infos sur un objet politique.
object_type_from_metadata()	Determine le type d'objet.
should_use_metadata()	Decide si un JSON doit entrer dans l'index.
infer_object_id()	Cree ou recupere l'identifiant d'objet politique.
normalize_author()	Nettoie les auteurs.
merge_authors()	Fusionne les auteurs venant de plusieurs documents.
document_record()	Cree un enregistrement de document lie.
scheduled_session_records()	Recupere les seances liees.
merge_metadata()	Fusionne les infos de plusieurs documents.
build_political_objects()	Construit les objets politiques.
political_object_rows()	Prepares les lignes SQL.
upsert_political_objects()	Insere/met a jour les objets en base.
rebuild_political_objects_index()	Reconstruit l'index complet.

app/political_object_normalization.py

Ce fichier met de l'ordre dans les statuts et dates des objets politiques.

Probleme qu'il resout:

Un objet politique a souvent plusieurs dates:

- depot
- premiere seance
- commission
- rapport
- decision
- reponse

Fonctions importantes:

Fonction	Role simple
NormalizedPoliticalObject	Structure finale normalisee.
normalize_status()	Traduit les statuts bruts en statut propre.
scheduled_deposit_date()	Trouve la date de depot planifiee.

Fonction	Role simple
<code>first_session_date()</code>	Trouve la premiere seance liee.
<code>latest_decision_session()</code>	Trouve la derniere decision.
<code>document_dates()</code>	Recupere les dates des documents.
<code>canonical_document_date()</code>	Choisit la date canonique du document.
<code>relation_date()</code>	Cherche une date selon un type de relation.
<code>commission_meeting_date()</code>	Trouve la date de commission.
<code>response_document_date()</code>	Trouve la date de reponse.
<code>normalize_dates()</code>	Choisit les dates finales importantes.
<code>normalize_row()</code>	Normalise un objet complet.
<code>upsert_normalization()</code>	Insere la normalisation en base.
<code>rebuild_political_object_normalization()</code>	Reconstruit toute la normalisation.

app/political_object_people_index.py

Ce fichier relie les objets politiques aux personnes.

Fonctions:

Fonction	Role simple
<code>load_people_lookup()</code>	Charge les personnes connues.
<code>load_political_objects()</code>	Charge les objets politiques.
<code>resolve_person_id()</code>	Essaie de retrouver l'identifiant d'une personne.
<code>relation_rows()</code>	Prepares les relations objet-personne.
<code>upsert_relations()</code>	Insere les relations en base.
<code>rebuild_political_object_people_index()</code>	Reconstruit ces relations.

app/political_object_documents_index.py

Ce fichier relie les objets politiques aux documents.

Fonctions:

Fonction	Role simple
<code>infer_relation_type()</code>	Determine le lien: document canonique, rapport, reponse, decision, etc.
<code>load_political_objects()</code>	Charge les objets politiques.
<code>relation_rows()</code>	Prepares les relations objet-document.
<code>upsert_relations()</code>	Insere les relations en base.
<code>rebuild_political_object_documents_index()</code>	Reconstruit ces relations.

app/financial_extraction.py

Ce fichier extrait des donnees financieres depuis les budgets.

Fonctions importantes:

Fonction	Role simple
<code>normalize_text()</code>	Nettoie le texte financier.
<code>parse_number()</code>	Convertit un montant texte en nombre.
<code>find_budget_files()</code>	Trouve les fichiers budget.
<code>extract_summary_tables()</code>	Extrait les tableaux de synthese.
<code>collect_amounts()</code>	Recupere des montants proches d'une ligne.
<code>extract_account_lines()</code>	Extrait les lignes de comptes.
<code>upsert_financial_data()</code>	Insere les donnees financieres en base.
<code>ingest_financial_budget_data()</code>	Pipeline complet d'extraction financiere.

Petits fichiers utiles dans app/

Fichier	Role simple
<code>config.py</code>	Lit la configuration: chemins, variables, backend.
<code>diagnostics.py</code>	Enregistre erreurs et interactions recentes.
<code>document_categories.py</code>	Normalise les categories de documents.
<code>eval_set.py</code>	Charge des questions de test.
<code>ingest.py</code>	Ancien/compatibilite: construit index JSON/SQLite et lance pipeline moderne.
<code>metadata_sync.py</code>	Synchronise les metadonnees JSON vers Postgres sans tout reindexer.
<code>sqlite_index.py</code>	Ancien moteur de recherche local SQLite.
<code>text_cleaning.py</code>	Corrige les accents/mojibake et nettoie le texte francais.

4. Les scrapers

Le dossier `scrape-la-tour-de-peilz/` contient les scripts qui vont chercher les donnees.

Ils suivent presque tous le meme modele:

```

fetch_text()
    telecharge une page HTML

collect_items()
    trouve les documents sur la page

download_and_extract()
    telecharge le PDF et extrait le texte

enrich...metadata()
    ajoute des metadonnees specialisees

main()
    lance tout le script

```

Scrapers principaux

Fichier	Role
<code>scrape_ordres_du_jour_2025_2026.py</code>	Recupere les pages de seances, ordres du jour et documents lies.
<code>scrape_proces_verbaux_2021_2026.py</code>	Recupere les proces-verbaux.

Fichier	Role
structure_proces_verbaux.py	Analyse les PV pour extraire presence, ordre du jour, votes, signatures, objets politiques.
scrape_motions_2021_2026.py	Recupere et enrichit les motions.
scrape_postulats_2021_2026.py	Recupere et enrichit les postulats.
scrape_interpellations_2021_2026.py	Recupere et enrichit les interpellations.
scrape_preavis_municipaux_2021_2026.py	Recupere les preavis municipaux et leurs rapports/decisions.
scrape_budgets_2021_2026.py	Recupere les budgets.
scrape_rapport_comptes_2021_2024.py	Recupere les rapports des comptes.
scrape_rapport_gestion_2021_2024.py	Recupere les rapports de gestion.
scrape_infos_municipalite_2021_2026.py	Recupere les infos de la Municipalite.
scrape_informations_diverses_2021_2026.py	Recupere les objets divers.
scrape_conseil_communal_institutionnel.py	Recupere la partie institutionnelle: membres, reglement, articles.

Fonctions typiques des scrapers

Tu verras souvent ces noms:

Fonction	Signification
fetch_text()	Telecharge une page web.
normalize_pdf_url()	Transforme un lien PDF relatif en URL complete.
safe_filename()	Cree un nom de fichier propre et compatible Windows.
extract_pdf_text()	Extrait le texte d'un PDF.
clean_html_text()	Nettoie du HTML pour en faire du texte.
clean_pdf_text()	Nettoie le texte extrait d'un PDF.
normalize()	Met un texte dans une forme comparable.
slugify()	Cree un identifiant lisible depuis un titre.
collect_items()	Trouve les documents a traiter.
download_and_extract()	Telecharge, extrait et ecrit .txt + .json.
main()	Point d'entree du script.

Scrapers motions / postulats / interpellations

Ces trois fichiers se ressemblent beaucoup.

Ils essaient de trouver:

- auteurs;
- parti;
- titre officiel;
- statut;
- document canonique;
- rapport;

- decision;
- lien avec une seance.

Fonctions recurrentes:

Fonction	Role
<code>parse_authors_from_listing()</code>	Recupere les auteurs depuis le titre sur la page web.
<code>clean_group_labels()</code>	Nettoie les mentions de groupes ou partis.
<code>extract..._authors()</code>	Recupere les auteurs depuis le contenu du PDF.
<code>extract..._object_title()</code>	Recupere le vrai titre de l'objet politique.
<code>infer_document_role()</code>	Determine si le PDF est l'objet original, un rapport, une reponse, une decision.
<code>extract_document_components()</code>	Liste les parties presentes dans le document.
<code>find_related...()</code>	Cherche le document canonique correspondant.
<code>mark_agenda_linked..._documents()</code>	Marque les documents qui sont lies a un ordre du jour.

scrape_ordres_du_jour_2025_2026.py

Ce fichier est particulierement important car il relie les seances aux documents.

Fonctions clefs:

Fonction	Role
<code>TextAndLinksParser</code>	Parseur HTML qui garde le texte et les liens.
<code>collect_session_pages()</code>	Trouve les pages de seance.
<code>parse_agenda_items()</code>	Extrait les points de l'ordre du jour.
<code>create_local_document()</code>	Cree un document local depuis un lien PDF.
<code>load_local_document_index()</code>	Charge les documents deja presents.
<code>load_canonical_object_index()</code>	Charge les objets politiques canoniques.
<code>canonical_for_link()</code>	Relie un PDF d'ordre du jour a son document canonique.
<code>add_session_to_canonical()</code>	Ajoute l'information de seance dans le document canonique.
<code>parse_session()</code>	Analyse une seance complete.
<code>write_session_files()</code>	Ecrit les fichiers de session.

structure_proces_verbaux.py

Ce fichier lit les PV deja extraits et essaie d'en faire des donnees structurees.

Fonctions clefs:

Fonction	Role
<code>extract_session_details()</code>	Trouve les infos generales de la seance.
<code>extract_signatures()</code>	Trouve les signatures.
<code>extract_agenda()</code>	Extrait l'ordre du jour du PV.
<code>extract_attendance()</code>	Extrait les presences/absences.
<code>vote_result_from_text()</code>	Interprete un resultat de vote.
<code>extract_previous_minutes()</code>	Trouve l'approbation des anciens PV.

Fonction	Role
<code>extract_decisions()</code>	Extrait les decisions.
<code>extract_political_objects()</code>	Extrait les objets politiques mentionnes.
<code>enrich_pv_metadata()</code>	Ajoute ces infos dans les metadonnees du PV.
<code>collect_pvs()</code>	Charge tous les PV.
<code>update_sessions_with_pvs()</code>	Relie les PV aux seances.

5. Comment lire le projet sans se perdre

Je te conseille cet ordre:

1. Lire ce document en entier une fois.
2. Ouvrir `app/ui.py`.
3. Chercher `answer_question`.
4. Suivre l'appel vers `cached_answer_question`.
5. Aller dans `app/structured.py` pour comprendre les reponses directes.
6. Aller dans `app/retrieval.py` pour comprendre la recherche.
7. Aller dans `app/answer.py` pour comprendre la reponse finale.
8. Lire `app/ingestion_pipeline.py` pour comprendre comment les documents entrent dans la base.
9. Lire les scrapers seulement apres, car ils sont plus longs et repetitifs.

6. Les mots importants du projet

Mot	Explication simple
RAG	Systeme qui cherche des sources puis demande au LLM de repondre avec ces sources.
Chunk	Petit morceau de document.
Embedding	Vecteur numerique qui represente le sens d'un texte.
Recherche vectorielle	Recherche par proximite de sens.
BM25 / mots-cles	Recherche classique par mots presents dans le texte.
Hybride	Recherche mots-cles + recherche vectorielle.
Metadonnees	Infos autour du document: titre, annee, auteur, type, URL.
Objet politique	Motion, postulat, interpellation, preavis, etc.
Source canonique	Document principal officiel d'un objet politique.
Document lie	Rapport, reponse ou decision liee au document principal.
Ingestion	Processus qui lit les documents et les indexe dans la base.
Reranking	Retri des resultats pour garder les plus utiles.

7. Le resume en une phrase

AI Riviera 2 est une chaine complete:

scraper -> nettoyer -> enrichir -> indexer -> rechercher -> repondre -> citer les sources

Si tu comprends cette phrase, tu as déjà la colonne vertébrale du projet.